HiRace: Accurate and Fast Source-Level Race Checking of GPU Programs

John Jacobson
Kahlert School of Computing
University of Utah
Salt Lake City, USA
john.jacobson@utah.edu

Martin Burtscher
Department of Computer Science
Texas State University
San Marcos, USA
burtscher@txstate.edu

Ganesh Gopalakrishnan
Kahlert School of Computing
University of Utah
Salt Lake City, USA
ganesh@cs.utah.edu

arXiv:2401.04701v1 [cs.DC] 9 Jan 2024

***Abstract*—Data races are egregious parallel-programming bugs on CPUs. They are even worse on GPUs due to the hierarchical thread and memory structure, which makes it possible to write code that is correctly synchronized within a thread group while not being correct across groups. Thus far, all major data-race checkers for GPUs suffer from *at least one* of the following problems: they do not check races in global memory, do not work on recent GPUs, scale poorly, have not been extensively tested, miss simple data races, or are not dependable without detailed knowledge of the compiler.**

Our new data-race detection tool, HiRace, overcomes these limitations. Its key novelty is an innovative parallel finite-state machine that condenses an arbitrarily long access history into constant-length state, thus allowing it to handle large and long-running programs. HiRace is a dynamic tool that checks for thread-group shared memory and global device memory races. It utilizes source-code instrumentation, thus avoiding driver, compiler, and hardware dependencies. We evaluate it on a modern calibrated data-race benchmark suite. On the 580 tested CUDA kernels, 346 of which contain data races, HiRace finds races missed by other tools without false alarms and is more than 10 times faster on average than the current state of the art, while incurring only half the memory overhead.

***Index Terms*—Data Races, Dynamic Analysis, GPU, Shared Memory, Concurrency**

I. Introduction

Data races are notoriously difficult to identify by manual code inspection or through the use of unit tests due to the complexity of modern memory models and the non-deterministic nature of race conditions. Data races are known to change results when ported to new platforms [1] and fundamentally produce unreliable binaries [2] because the assumptions under which a compiler optimizes code are invalidated [3] by data races.

GPU programs are in many ways more susceptible to data races due to the large number of concurrent threads and the availability of multiple layers of shared memory through which these threads may interact. Although GPU languages provide synchronization primitives and atomics to enforce orderings on memory actions, their use is not always intuitive and they have significant performance costs. Developers often utilize complex code patterns to avoid synchronization overheads, making the code difficult to reason about. This has resulted in many attempts at creating GPU data-race detection tools (see Table I), yet there are still no good practical solutions available even for the most widely used GPU languages.

Most of the GPU data race checkers developed to date in Table I are not in working condition, thus leaving the community with two options (both of which are specific to NVIDIA GPUs), namely iGUARD [4] and Compute Sanitizer [5]. However, Compute Sanitizer does not detect global memory races—a serious omission considering that many GPU programs concurrently access global memory to reduce data movement. iGUARD, and its future evolution, heavily depends on support by the NVBit tool [6], does not seem to be actively maintained, and is not functional on NVIDIA architectures released since its publication.

In this paper, we present HiRace, a practical and efficient dynamic data-race checker that aims to serve the needs of a vast majority of today's HPC and ML practitioners. HiRace works at the source-level by automatically instrumenting code through Clang's source rewriting API [13], part of the widely adopted and actively maintained LLVM project. Source-level instrumentation allows us to avoid reliance on compiler contracts and variability due to compiler optimizations. Functionally, HiRace is designed with a focus on the most common GPU programming patterns, providing significant improvements in completeness, run time, and memory overhead for codes utilizing GPU synchronization primitives.

Design Novelties

Single Access Record: The most significant novelty in HiRace is based upon the observation that a fixed-size accessor record (elaborated in Section III) is sufficient to encode all necessary information about *all* prior accesses to identify races. This design ensures that the memory overhead is constant and small. In comparison, other designs maintain multiple accessor records to dynamically reconstruct the access history. This requires significantly more memory, and repeated accesses to a single location can cause an accessor record to be evicted due to capacity conflicts, causing race omissions (discussed further in Section V).

Explicit State-Machine: HiRace achieves a modest size (constant space per memory location) access history through the use of an explicit finite-state machine that encodes access histories compactly (5 bits of state per tracked memory loca-

Tool Name	Status	Generality, Usability
GPUVerify [7]	Not Maintained	Static analyzer, limited applicability
GKLEE [1]	Not Maintained	CUDA as of 2016
PUG [8]	Not Maintained	Limited CUDA subset
CURD [9]	Not Released	CUDA
Barracuda [10]	Not Maintained	CUDA
LD [11]	Never Released	Useful when tracking states affordable
iGUARD [4]	Not Maintained	NVIDIA SASS binary instrumentation
Faial [12]	Recent release	Limited to a dozen or so straight-line code blocks without conditionals or loops
Compute Sanitizer [5]	Maintained	NVIDIA proprietary, No global-memory data race detection

TABLE I
EXISTING GPU DATA-RACE CHECKING TOOLS

tion). This is achieved in part by equating threads to thread groups following the GPU concurrency hierarchy as elaborated in Section III.

Correctness: Having an explicit state-machine also allowed us to develop a Murphi [14] formal reference model, which we tested with a model-checker to match against HiRace's behavior. This exercise gives increased confidence in the correctness of our design, and aids in extending our methodology to new and more complex memory models (a similar verification exercise was reported in [15]).

To further increase our confidence in HiRace's race-checking completeness, we test it against *calibrated* benchmarks from the Indigo suite [16], which contains thousands of GPU codes that are labeled as bug-free or buggy, including which bugs they contain. Indigo benchmark codes contain irregular graph algorithms, which exhibit data-dependent memory accesses and control flow. Such behavior makes programs harder to debug because even buggy codes will execute correctly for inputs that happen to yield (1) control flow that avoids the problematic code sections or (2) memory-access patterns that exclude the problematic data dependencies. In other words, only certain inputs may trigger the software defects present in the code. Moreover, the thread timing in parallel programs similarly only triggers software defects in some but not all executions of a program, even when using the same input. Together, this makes detecting bugs in irregular parallel programs particularly challenging, and we believe makes a suite like Indigo an ideal test target for verification tools like HiRace.

We observe that HiRace's reporting conforms to Indigo's calibration and also identified an unintended data race within some Indigo benchmarks. We find that **HiRace can be 30× to 50× faster than state of the art—with median speedups of at least 7×** (Figure 5).

This paper makes the following main contributions. HiRace

- employs an innovative state-machine-based design,
- detects more races than the state of the art in dynamic GPU data-race detection,
- is on average 10× faster and scales to much larger programs compared to state of the art,
- has been tested against a calibrated set of benchmarks,
- uses source-level instrumentation based on Clang, and
- has been verified with the Murphi model checker.

HiRace will be submitted to the artifact committee and will be open-sourced after the end of the anonymity phase.

Section II provides background on shared-memory race checking in general and GPU data-race checking in particular. Section III presents a systematic walk-through of HiRace's design. Section IV describes our experimental methodology. Section V presents our result evaluation. Section VI provides concluding remarks.

II. BACKGROUND

Data-race checking is a topic with a long history [17]. In this section, we outline important background on data races and CUDA programming paradigms. We also present prior work and contrast it with HiRace.

A. Data Races

A parallel program has a data race if multiple threads access the same memory location, at least one of the accesses is a write, and the accesses are concurrent (more precisely, are not ordered by some *happens-before* [18] relation). The definition of a data race has been refined to its present form based on a precise notion of happens-before [2], which is supported by venerable tools such as FastTrack [19] and Google's Thread Sanitizer [20]—primarily addressing shared memory (POSIX-like) threads.

Consequently, detecting data races requires first identifying all memory accesses, particularly those that are *conflicting* (accesses to the same location from different threads), and second determining whether any conflicting accesses thus identified are concurrent. While doing so, data-race-detection tools aim for a balance between two properties: reporting all races ("no omissions", or *completeness*) and NOT reporting any non-races ("no false alarms", or *soundness*). There are two primary approaches to making these determinations: *static* and *dynamic* analysis of a program.

Static analysis is performed by parsing program instructions without executing them. This generally involves utilizing some encoding of the program semantics that model the relevant behaviors of the code being analyzed. Static analyses have the benefit of complete program knowledge, but, unfortunately, suffer from the undecidability of many program properties (for example, the halting problem). As a result, static analysis

methods inherently suffer from reporting false positives. This is a significant issue in practice as it may add cost either in the form of code review time or unnecessary development time and overhead in programs, and they may erode confidence in the tool's other error reports, causing users to ignore real errors. For this reason, our work on HiRace focuses on dynamic analysis.

Dynamic analysis is performed by monitoring the underlying program at run-time. Doing so avoids the burden of determining reachability of instructions since all executed instructions can be directly witnessed. However, dynamic methods are restricted to analyzing program instructions that are observed in a given test execution. Any conditional code paths that are not explored by a given configuration cannot be tested, leaving regions of code unanalyzed. This problem can be alleviated by testing the code with multiple inputs that elicit the various possible program behaviors.

Many modern tools employ a combination of these methods, such as Archer [21] and GKLEE [22] (discussed below).

B. CUDA Shared-Memory Concurrency

CUDA devices present large numbers of threads to the user, which are organized into hierarchical structures to be dynamically scheduled and executed on a number of streaming multiprocessors (SMs). Each SM has several SIMT vector units. Modern GPUs have over 100 SMs on a single device, each capable of hosting over 1,000 resident threads.

Logically, individual threads are arranged into *thread blocks*, which are executed on a single SM. The threads of a thread block are arranged into *warps*, which are executed on a single SIMT unit. Device memory is composed of several spaces (or *scopes*) with differing accessibility. *Global memory* is accessible by all threads on the device, whereas *shared memory* is allocated to individual thread blocks and is accessible only to threads within the same thread block.

For managing this thread hierarchy, CUDA provides a set of block-scope synchronization primitives (*barriers*) as well as a set of warp-scope synchronization primitives.

The most basic block scoped barrier is `__syncthreads`. Informally, any thread that executes a `__syncthreads` instruction will wait for all threads within the same thread block to execute the same `__syncthreads` before proceeding to its next instruction. There are variants of this primitive that provide the same guarantee but also perform a thread-block-wide reduction operation and broadcast the result to all threads in the block.

The most basic warp-scoped barrier is `__syncwarp`. It provides the same functionality as `__syncthreads` but is limited to the threads within the same warp. Additionally, `__syncwarp` takes a *mask* argument that restricts the participating threads to only those specified in the mask (which must all be members of the same warp). Like for the block-level barriers, there are additional warp synchronization primitives that allow thread communication alongside the synchronization effects.

A barrier-only concurrency structure without data-dependent conditionals enjoys the so called *two-thread theorem* [8], [23], which is employed in GKLEE and GPUVerify. It states that, if there is a race, it can be discovered within the scope of just two threads.

C. Other Related Work

The CPU-based FastTrack algorithm for Java pioneered many ideas in dynamic race checking that can still be found in current checkers. Specifically, the idea of maintaining multiple access records to each word in memory originated in FastTrack and is utilized in ThreadSanitizer and its derivatives, as well as in IGUARD. FastTrack showed formally that, while writes in a data-race-free program are totally ordered, reads are not. As a result, the algorithm requires an unbounded read history for each word to be complete. In practice, the access history generally contains only 2 or 4 prior accessors in tools that implement this algorithm on the CPU. IGUARD is only able to store a single prior accessor due to the size of each access record on the GPU. In contrast, HiRace summarizes this history by tracking *groups of threads* as a single identifier, along with a small state tag (discussed in Section III). This allows us to achieve a much smaller memory footprint as well as a more complete analysis since HiRace does not need to evict prior access history.

In the GPU arena, race-checkers based on static analysis (GPUVerify, PUG), dynamic analysis at the source level (CURD, LD), dynamic symbolic execution (GKLEE), and dynamic binary-level analysis (IGUARD, Barracuda) have been developed, as summarized in Table I.

Compute Sanitizer is NVIDIA's proprietary correctness tool suite for CUDA programs and includes a *RaceCheck* tool. The details of these tools' implementation are not publicly available, but since it is included within the standard CUDA toolkit, it is the most widely available and easy to use race-detection tool currently available for GPUs. Unfortunately, Compute Sanitizer does not check global device memory for data races. Moreover, we found that it also misses data races *in shared memory*, which HiRace is able to identify (discussed in Section V).

IGUARD [4], an evolution from ScoRD, the prior reported in [24]. It is an open-source dynamic binary-instrumentation-based data-race-detection tool for CUDA programs. To our knowledge, IGUARD represents the current state of the art in GPU data-race detection as it outperforms other available tools in most categories. It utilizes the NVBit [6] API to dynamically instrument CUDA programs. It is simple to use as the compiled binary can be loaded as a shared library at runtime when executing the test program, and it supports both global and shared memory. The design of IGUARD is based on the same logic presented in FastTrack but optimized for execution on CUDA devices. Unfortunately, this design relies on an unbounded access history for completeness, and, due to the scale of CUDA programs, only one prior accessor can be tracked in 16 bytes of metadata per monitored word of memory. More importantly, the code base has not been

updated since its publication and newer generations of CUDA devices are no longer supported by IGUARD. The primary distinction between HiRace and IGUARD is that the latter was designed with a focus on traditional CPU synchronization patterns (particularly on automatic lock inference), which are less common on GPUs, whereas HiRace focuses on the most common patterns, which utilize barrier primitives or atomics.

III. Design of HiRace

Our design is based around the built-in synchronization primitives available in GPUs (such as CUDA's *__syncthreads*) as opposed to focusing on the full generality of CPU parallel programming patterns. Our method does not preclude the handling of fine-grained thread synchronization, but we focus on optimizing the most common and impactful forms of thread communication in GPUs.

We achieve this by relating all possible thread access histories to a single memory location. For example, if a thread reads the same memory location multiple times, we need not record each access separately. A single thread ID and a flag indicating a read access by that ID suffices. Going a step further, in a program with only 2 threads, if both threads concurrently read the same memory location (in any order and any number of times), it suffices to only track the ID of either accessor along with a flag indicating that *multiple* threads have read this location.

In the general case, this would require 2^T flags to track all possible thread relations between T accessing threads. However, due to the hierarchical and symmetric nature of the shared memory scopes of GPUs, instead of tracking individual threads at every memory interaction, *we only track the hierarchical scope shared by the accessing threads*. These scopes provide naturally symmetric work groups that are uniquely determined by the ID of any thread within the group (i.e., certain bits of a global thread ID). As such, a single identifier (the last accessing thread ID) and a flag representing the scope of access (i.e., thread-block scope) are sufficient to characterize a large class of unbounded access histories (arbitrary reads from multiple threads within the same thread-block).

HiRace is based on a finite state machine (FSM) that encodes the happens-before logic introduced earlier. Whereas the FSM is tailored to the current CUDA programming model, it can easily be extended to other models and deeper hierarchies. The HiRace FSM has 25 states and 1200 transitions between states. They are sufficient to describe the interactions of all atomic and non-atomic reads and writes to a single memory location as well as all needed synchronization history from warp-scoped and block-scoped barrier primitives.

Rather than exhaustively detailing each state of the FSM (whose complete specification will be made publicly available), this section incrementally develops similar state machines for simpler concurrency models. To this end, we present two scenarios that introduce the key ideas behind our approach: how the FSM performs atomic updates, how our source instrumentation works, and how we succinctly express the FSM transitions.

A. Simple Scenarios

```
1  __global__ void race_noSync(int* data) {
2    int i = threadIdx.x + blockIdx.x * TPB;
3    int val = data[0];
4    data[0] = i + val;   // race!
5  }
```

Listing 1. Read-write data race

First, we consider the set of CUDA programs containing only non-atomic reads and writes to a single memory space (global memory) that use no form of synchronization. Listing 1 provides an example of such a program.

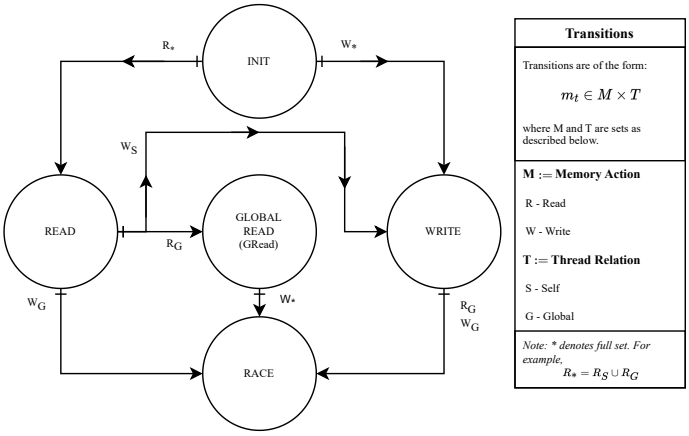


Fig. 1. State machine for code without barriers; any unmentioned outgoing transition from a state means we stay in the same state—a "self loop"

We claim that the FSM in Figure 1, along with a single prior accessor ID, is sufficient to track all data races that may occur in such programs along a single control-flow path.

To monitor the $data$ array for races, we create a $shadow[k]$ entry for each array element $data[k]$, as shown in Figure 2. Each $shadow[k]$ entry maintains the following information: the block-scalar clock (BC), the warp-scalar clock (WC), the ID of the thread performing the *most recent access* (TID), and the state of the FSM from Figure 1 ($State$).

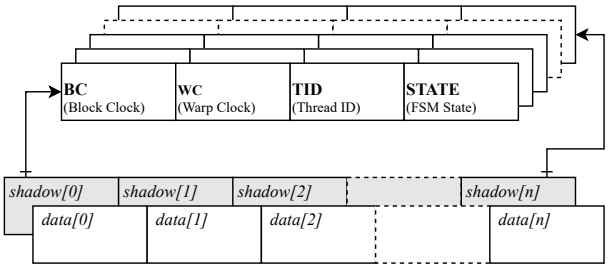


Fig. 2. Shadow information used by HiRace

Initially, all $State$ values are set to INIT, indicating that no memory accesses have occurred yet. Let us assume the thread organization shown in Figure 3. We label the eight threads T_{000} to T_{111} to indicate their block, warp, and TID. For example, the "red" thread is T_{011}.

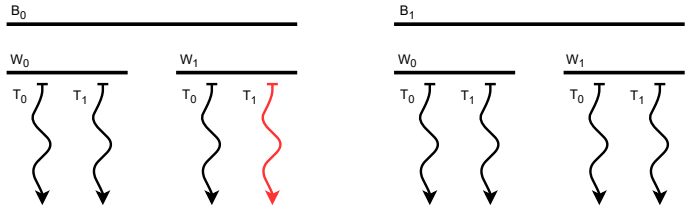


Fig. 3. Grid of CUDA threads assumed in Listing 1 (for which we ignore B0 and B1) and Listing 2 (for which we consider B0 and B1)

In the scenarios we describe, all CUDA threads accessing $data[k]$ atomically copy, due to our source instrumentation, $shadow[k]$ into $oShadow$, which is the tuple $\langle oBC, oWC, oTID, oState\rangle$[1]. Then, they determine the new tuple $\langle nBC, nWC, nTID, nState\rangle$ to write back to $shadow[k]$ in a lock-free manner using an ATOMICCAS instruction. Only one thread will succeed; the "losers" re-obtain a copy of $shadow[k]$ and try again. To illustrate this process, let us walk through a few specific scenarios.

Scenario 1: Suppose T_{011} (the "red" thread) and T_{100} concurrently execute Line 3 in Listing 1. Suppose further that T_{011} is the winner. At this point, the instrumented code transitions the $State$ from INIT to READ (the transition is labeled R_*, meaning "any read by any thread"). Since T_{100}'s update attempt fails, it re-reads $State$ and finds it to be READ with a different TID. Thus, it advances the state to GREAD—meaning **G**lobal read—since this memory location was previously read by a different thread. If thread T_{011} executes Line 4 next, the state machine follows the W_* arc (meaning "any write at all"), transitioning the state to RACE.

Note that there is no obligation for the instrumented code handling T_{100} to update $shadow[k]$ before T_{011} executes Line 4. In other words, *there is no requirement that a location access and its instrumented code execute atomically.*[2]

Of course, it is quite possible that only T_{011} has updated $shadow[k]$ when T_{011} reaches Line 4, in which case it updates the state from READ to WRITE following the W_S transition, which stands for **S**ame-thread write. If the instrumented code for T_{100} now updates $shadow[k]$, it enters the RACE state since any read or write access by a different thread is a race.

```
1  __global__ void race_blockSync(int* data)
2  {
3    int tid = threadIdx.x;
4    int val = data[0];
5    __syncthreads();
6    if (tid > 0) {
7      data[tid - 1] = tid;   // race!
8    }
9  }
```

Listing 2. Read-write data race with syncthreads (block-scope barrier)

[1]Prefix o indicates "old" values, i.e., the values stored by a prior accessor, whereas prefix n denotes the "new" values determined by the current accessor.

[2]This is also important for performance.

Scenario 2: For this scenario, we consider the program in Listing 2, which contains a block-scoped syncthreads barrier. To explain the state-machine activities in the presence of barriers, we extend the notation on how reads and writes are subscripted. The subscripts are of the form $\{S, T\}$, where S is the synchronization status, which is either Us ("unsynchronized") or Bs ("block-synchronized"), and T is the thread relation, which now has a B ("block") option in addition to the former S ("self") and G ("global"). The new thread relation B indicates that a barrier has "block-synchronized" the access (indicated by differing block scalar clock, BC, values).[3] The wildcard "$*$" means any applicable letters.

Consider the scenario of T_{011} (the "red" thread of Fig. 3) executing Line 4 in Listing 2, where it reads location 0 of the data array. This access updates the state in $shadow[0]$ to READ, as indicated by the wildcard-subscripted $R_{\{*,*\}}$ transition in Fig. 4. Suppose T_{000}, another thread in the same block, now executes Line 4. This access updates $shadow[0]$ to BREAD (Block Read)—a new state—that captures the fact that two threads from the same block scope have read this location asynchronously, as indicated by the label $R_{\{U_s,B\}}$. As long as other threads from the same block execute Line 4, $shadow[0]$ remains in the BREAD state. As soon as a thread from a different block, for example T_{110}, executes Line 4, we promote $shadow[0]$ to GREAD ("global read") as a read has been performed in the "global scope" ($R_{\{*,G\}}$).

Once we are in the GREAD state, any type of write by any thread ($W_{\{*,*\}}$) results in a RACE as this memory location has witnessed a read from at least two blocks. This is important because syncthread barriers do not synchronize threads across blocks, so any writing thread must be executing asynchronously to at least one of the prior readers.

Instead of thread T_{110} performing the aforesaid access, let $shadow[0]$ remain in BREAD and let the red thread T_{011} execute the `__syncthreads()` call on Line 5. Because of the barrier semantics, all threads must cross the barrier before they can execute any following instruction. Therefore, eventually, the thread-private block-scalar clock (BC) updates to 1 in all threads in this block. At that point, a write by any thread in block B_0 sends the line to the WRITE state as indicated by the transitions $W_{\{B_s,S\}}$ and $W_{\{B_s,B\}}$. Even so, T_{110} eventually performs a read of this location, thus transition to the RACE state. The remaining transitions of the FSM in Fig. 4 may be similarly explored by considering arbitrary interleaving of supported operations.

B. Further Extension

In a similar manner, the HiRace FSM is extended beyond what is shown in Fig. 4 to support atomics and warp-scoped synchronization primitives. This expands the full state machine to 25 states and 1200 transitions.

Atomic operations are represented as a third class of memory action (beyond Read and Write) and require several new

[3]The size of these clocks is configurable, and race detection is discontinued with a warning if a clock overflows (after reporting any previously identified races).

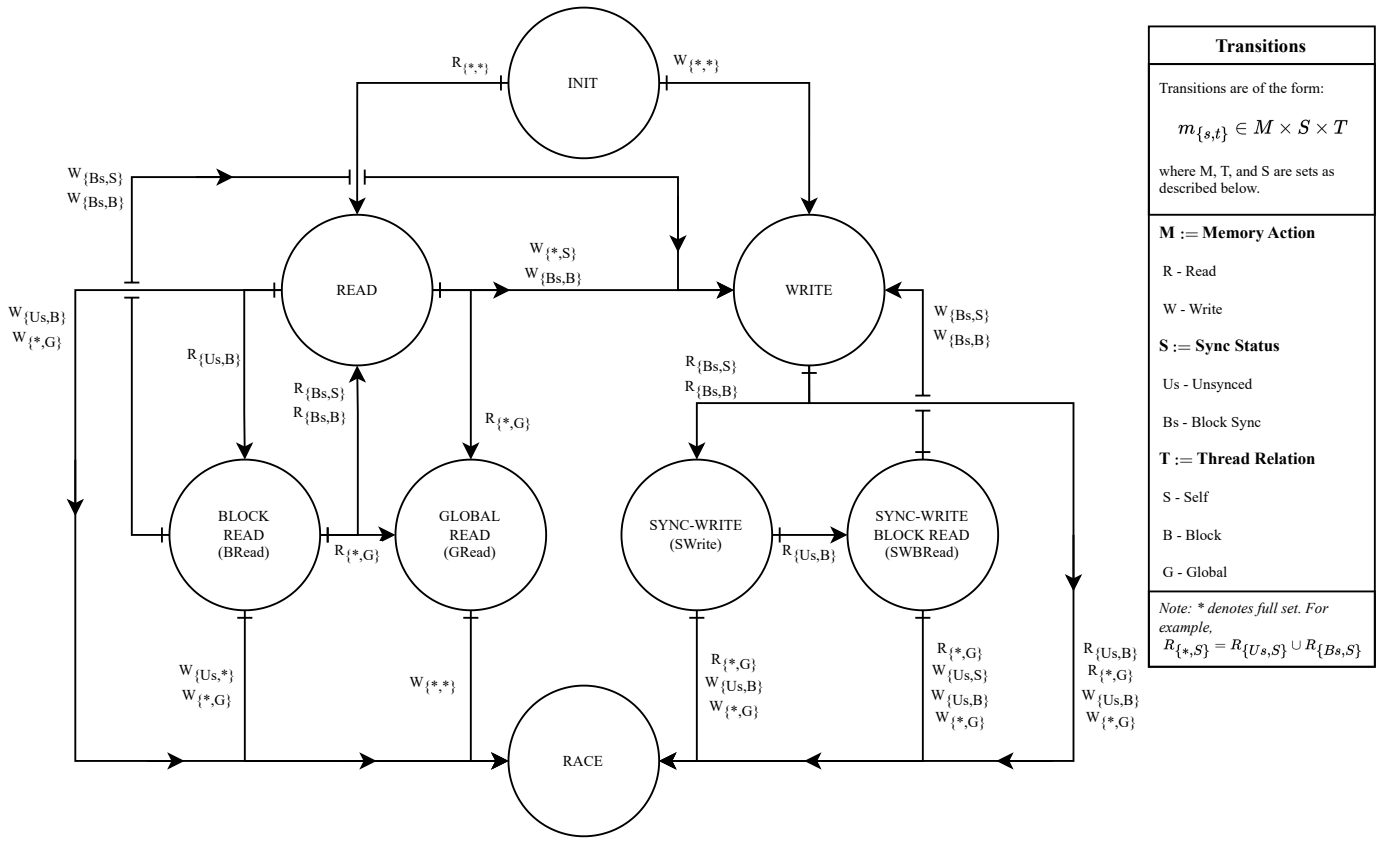


Fig. 4. State-machine for code with block synchronization; any unmentioned outgoing transition from a state means we stay in the same state

states that describe the corresponding access histories (for example, a synchronized atomic access is distinct from a synchronized write access).

Warp-scoped barriers similarly require expanding the transition function with a new synchronization status and new thread relations. They also require additional states to capture the warp-scoped synchronization history while retaining the higher-scoped synchronization history.

This section only provides an informal view of how our FSM works. To formally prove its correctness, we verified it with an exhaustive state-space search using a Murphi [14] model.

IV. Implementation

HiRace is implemented primarily as a template library, supported by Clang's AST tools for automated instrumentation. Its configuration is exposed through a *toml* interface, allowing the user to specify a number of features, such as which kernels and variables to monitor for races, which thread groups to monitor, and bounds for shadow value metadata to reduce memory overhead.

A. Memory Access Monitoring

To capture memory accesses, HiRace utilizes a templated wrapper class to monitor user data structures. For each shared memory variable, a shadow value data structure is allocated. Both the original shared memory variable and the shadow value pointer are tracked within a thread-local instance of the HiRace wrapper. The wrapper then overrides relevant operators (for example, the subscript or array index operator `operator[]`) to intercept memory access events and update the associated shadow value transparently. This approach also requires overriding other relevant functions, such as atomic functions and synchronization primitives.

This methodology allows flexible management of the underlying test data structures as well as their associated shadow structures and thread metadata. Individual threads or test data addresses may have their instrumentation enabled or disabled dynamically, and the memory footprint can be reduced by tracking only representative threads for user-defined symmetric work groups instead of tracking all threads.

B. Race Detection

The core of HiRace's detection logic is embedded in the shadow state update algorithm (Algorithm 1) built around the state machine described in Section III.

As previously described, our FSM contains 25 states and 1200 transitions. It requires only 5 bits of memory to record the FSM state per monitored memory address, which is combined with other metadata (as described in Fig. 2) resulting in 8 bytes of memory per address by default (though this is configurable and can be reduced). In contrast, the state of

Algorithm 1 Lock-Free Shadow Value Update

Require:

1: $sAddr$: Shadow value address
2: $access$: Access type (read/write/atomic)
3: tid : Global thread ID
4: bc : Block-scope scalar clock
5: wc : Warp-scope scalar clock
6: **procedure** UPDATESHADOW($sAddr, access, tid, bc, wc$)
7: **repeat**
8: $oShadow \leftarrow$ ATOMICREAD($sAddr$)
9:
10: $\langle oState,$
11: $oTid,$
12: $oBC,$
13: $oWC\rangle \leftarrow$ UNPACKSHADOW($oShadow$)
14:
15: $tRel \leftarrow$ COMPARETIDS($tid, oTid$)
16: $sRel \leftarrow$ CHECKSYNC(bc, oBC, wc, oWC)
17:
18: $trans \leftarrow$ GETTRANS($oState, access, sRel, tRel$)
19: $nState \leftarrow$ STATEMACHINELOOKUP($trans$)
20:
21: $nShadow \leftarrow$ PACKSHADOW($nState, tid, bc, wc$)
22: **until** ATOMICCAS($sAddr, oShadow, nShadow$)
23: **end procedure**

the art uses 16 bytes per monitored address to track 2 prior accessors with no ability to reason about any earlier accesses in general.

When a thread accesses a monitored memory address through the HiRace wrapper, it calls Algorithm 1. This causes the thread to (atomically) check the metadata for the memory location it is accessing, compare it against its own current metadata, and use the result to update the state to a new shadow value (again atomically).

In more detail, the first step is entering the lock-free loop in Line 7 of Algorithm 1. Any accessing threads may concurrently compare the old shadow value ($oShadow$) to their own metadata, but a thread may only store its new shadow value ($nShadow$) if it verifies, via the `atomicCAS`, that $oShadow$ has not changed. If any other thread updates the shadow value in the meantime, the current thread detects the change, starts over, and re-calculates $nShadow$ relative to the newly read $oShadow$. Thus, all shadow-value accesses in shared memory are atomic and guaranteed to be single transitions represented by our FSM.

Since most of the race-detection logic is contained within the FSM, the amount of computation in the `atomicCAS` loop is minimal. Each shadow value is a bit array as outlined in Fig. 2. $UnpackShadow$ splits the shadow value into individual "old" metadata values from the prior access. $CompareTids$ and $CheckSync$ compare both old and new metadata values to determine the Thread Relation and Sync Status labels as described in Fig. 4.

The HiRace FSM is encoded as a flat array indexed by a concatenation of $oState$ and the transition labels $m_{\{s,t\}}$ shown in Fig. 4, which map to a new state. $GetTrans$ creates an integer index into the state-machine array using bit operations to combine $oState$ with the current Memory Action ($access$), Thread Relation, and Sync Status. Afterwards, $StateMachineLookup$ is just an array access to index $trans$.

Finally, the newly calculated state is concatenated with the current metadata, and the thread attempts to store the result.

C. Source Instrumentation

To facilitate instrumentation with our wrappers, we provide a standalone Clang tool for AST parsing and rewriting. The Clang LibTooling C++ interface facilitates parsing source code into AST as well as walking and rewriting the AST and/or source code through a robust API.

As HiRace is a header library designed to wrap around existing data structures and override relevant CUDA language primitives, it can be utilized through manual instrumentation fairly easily. However, the HiRace Clang tool automates this instrumentation. The Clang AST allows us to mechanically identify and wrap monitored data pointers and to allocate and deallocate necessary metadata. Users may simply execute this tool on their source code to generate instrumented source, which may be compiled and executed immediately or analyzed by hand or static tools if necessary.

V. EVALUATION

A. Experimentation Methodology

Our experiments were executed on a system with the following configuration:

- CPU: AMD Ryzen 5 3600
- GPU: NVIDIA Geforce RTX 2070 Super
- OS: Ubuntu 22.04
- CUDA version: 11.7

B. Section: Indigo Benchmarks

We use programs and inputs from the recently released Indigo benchmark suite [16] to drive our experiments. Unlike conventional suites, Indigo contains scripts and configuration files that allow the user to generate the desired codes and inputs.

There are 14 graph generators to choose from for creating parameterizable inputs of arbitrary size. There are also 21 CUDA kernel generator patterns, which generate up to 580 distinct CUDA programs operating on a given input data type.

More importantly for us, the code generator is not only able to synthesize hundreds of different versions of common graph-processing code patterns but also to systematically insert various bugs. We make extensive use of this latter capability as it enables us to control precisely which bugs, if any, are present. This provides a reliable ground truth valuation, which is typically not available with other suites or codes. Each benchmark kernel comes paired with a sequential implementation of the same algorithm as a baseline verification of the benchmark's correctness (or lack thereof).

Input Graph	Number of tests	HiRace Races Found	HiRace Races Missed	iGUARD Races Found	iGUARD Races Missed	Compute Sanitizer Racecheck Races Found	Compute Sanitizer Racecheck Races Missed	Sequential Comparison Races Found	Sequential Comparison Races Missed
DAG_100n_200e	580	346	0	298	48	92	254	301	45
DAG_200n_400e	580	346	0	298	48	92	254	301	45
DAG_5n_5e	580	182	164	118	228	92	254	44	302
counterDAG_200n_1000e	580	346	0	298	48	56	290	282	64
counterDAG_5n_5e	580	295	51	148	198	92	254	194	152
power_law_200n_1000e	580	346	0	297	49	128	218	306	40

TABLE II

Number of Indigo benchmark codes tested by each tool on the listed input.
Here, the column "Sequential Comparison" indicates benchmarks for which sequential execution and parallel execution did not produce matching results—indicating that the parallel execution exhibited a data race. These results are discussed further in Section V-D.

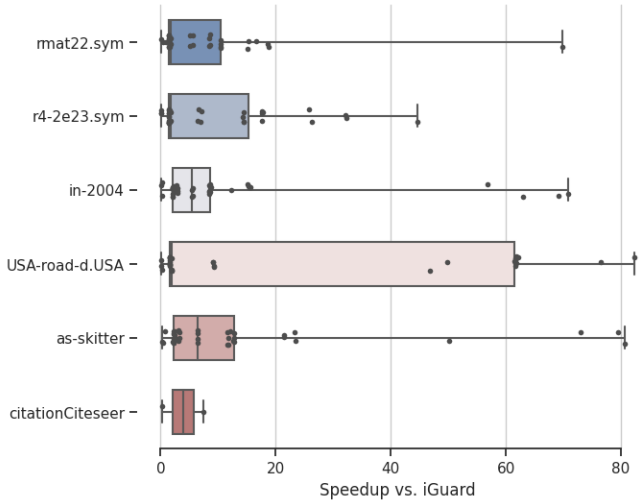


Fig. 5. HiRace speedup vs. iGuard. Each point represents execution of one Indigo benchmark kernel on the associated input graph.

Graph	Type	Vertices	Edges
rmat16.sym	RMAT	65,536	967,866
internet	Internet topology	124,651	387,240
USA-road-d.NY	road map	264,346	730,100
citationCiteseer	publication citations	268,495	2,313,294
amazon0601	product co-purchases	403,394	4,886,816
2d-2e20.sym	grid	1,048,576	4,190,208
in-2004	web links	1,382,908	27,182,946
as-skitter	Internet topology	1,696,415	22,190,596
rmat22.sym	RMAT	4194304	65,660,814
r4-2e23.sym	random	8,388,608	67,108,846
USA-road-d.USA	road map	23,947,347	57,708,624

TABLE III

REAL-WORLD GRAPH INPUTS

C. Results

For our tests we, used Indigo to generate 580 CUDA kernels, of which 346 contained data races. We also generated a number of small input graphs for testing the accuracy of HiRace, and used 6 graphs ranging between 5 and 200 nodes to evaluate both HiRace and iGUARD. The results are displayed in Table II.

We also compared our results to Compute Sanitizer's RACECHECK tool, but since it only checks for CUDA block-shared-memory races, there were relatively few examples to compare against, and both iGUARD and HiRace found all block-shared-memory races identified by Compute Sanitizer.

We found that, on the 3480 Indigo benchmark comparisons between iGUARD and HiRace, neither tool provided any false positives. While iGUARD performed quite well in identifying data races given the constraints of classical race detection algorithms, HiRace was able to find 100% of the injected data races in the Indigo benchmarks (when provided with inputs that exhibited the injected data race).

Further, we tested the same codes on eleven large scale real-world graph inputs (as described in Table III) to test performance, as shown in Figure 5. iGUARD struggled with large grid dimensions, and the timing results were very sporadic (occasionally requiring multiple hours, yielding over 1000x slowdown over baseline code execution). We were able to compare 441 executions across these large graphs to iGUARD, which demonstrated an overall average overhead of more than 30x slowdown, against HiRace's average of less than 3x slowdown across the same code/input combinations.

With HiRace's low overhead, we were able to more thoroughly test its performance against baseline codes. Across 5,105 executions (464 Indigo codes across the 11 large graphs in Table III), HiRace demonstrated 7.5x average slowdown, and 1.08x median slowdown (while still identifying 100% of the injected data races, with no false positives).

HiRace also has a few outliers in runtime, primarily due to its race reporting method, which requires significantly more I/O on some code patterns to report races on unique memory addresses. We intend to address this (and provide configuration options for reporting) in the future.

For additional perspective, we compared the same three tools on some[4] benchmarks from the Rodinia [25] suite. Rodinia contains a number of CUDA codes representing common GPU code patterns. The tested programs and runtime performance are described in Figure 6.

SRAD [26] (Speckle Reducing Anisotropic Diffusion) is a diffusion algorithm for ultrasonic and radar imaging applications. Although it utilizes relatively little block-scoped shared memory, Compute Sanitizer is several times slower than HiRace, exhibiting more than 76x slowdown of the

[4]The Rodinia codes are old, and most of them contain features that are no longer supported by recent versions of the nvcc CUDA compiler.

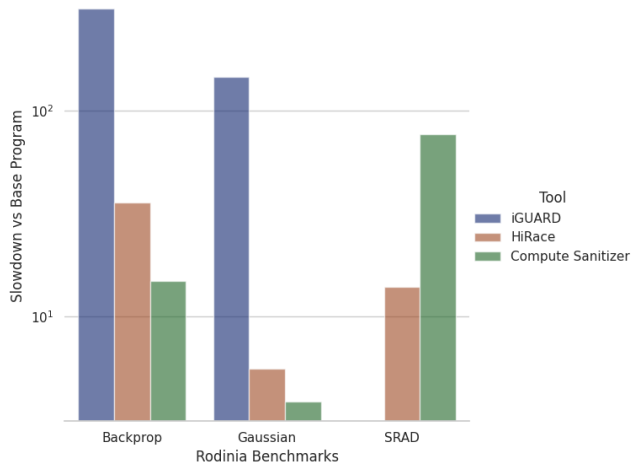


Fig. 6. Race-Detection Slowdown of Rodinia Benchmarks.

```
1  __global__
2  void multiRead_Race(int* data, int
       len)
3  {
4    int tid = threadIdx.x;
5    int local_sum = 0;
6
7    if (tid < len - 2) {
8      local_sum = data[tid]; // evicted
9      local_sum += data[tid + 1];
10
11     data[tid+1]=local_sum; // race!
12   }
13 }
```

Listing 3. Finite-history litmus test

base program. HiRace by comparison has less than a 14x slowdown. IGUARD, unfortunately, is unable to run this code.

Backprop is a machine learning back-propagation algorithm used to train neural-network layers. This algorithm consists of forward and backward phases updating a network of user-defined depth. Compute Sanitizer performs much better on this program with less than 15x slowdown. HiRace incurs 36x slowdown, whereas IGUARD has over 300x slowdown. We believe this is due to IGUARD's use of CUDA's Unified Virtual Memory (UVM).

The Gaussian program is a simple Gaussian Elimination algorithm for 2D matrices. Compute Sanitizer incurs less than 4x slowdown as the program does not utilize block-shared-memory, so there is nothing for it to monitor. HiRace is able to check global memory usage with less than 6x slowdown. IGUARD also struggles to scale with higher matrix dimensions on this kernel, showing over 145x slowdown.

Lastly, using our default shadow value footprint of 8 bytes per tracked memory address, we performed all of our tests with less than half of the memory overhead of IGUARD. In a real application, we would be able to reduce this footprint further by configuring HiRace's settings to more efficiently track the parallel patterns of a given test application.

D. Discussions

One of the largest weaknesses of dynamic race detection tools is the need for arbitrary length access histories. Maintaining a full access history is infeasible, so these tools must balance the history length against memory overheads.

Listing 3 demonstrates a simple pattern in which 2 distinct threads read the same location in succession. Though some CPU race detection tools can afford longer histories, which may capture both reads, IGUARD in particular relies on a single prior reader and a single writer (due to the higher cost of GPU thread metadata). Because of this, IGUARD fails to identify any data race in Listing 3's kernel.

For example, a read from $data[1]$ by thread 1 on Line 8 is lost if thread 0 later reads the same location on Line 9. Thus, when thread 0 writes to $data[1]$ on Line 10, although the access is concurrent to the read by thread 1 on Line 5, this history has been lost (in this particular thread schedule) and the race goes undetected. Note that this example can easily be extended to any finite history depth, thus avoiding detection on any prior race checker that is based on a finite history.

With HiRace's FSM design, any number of reads and any scheduling may be represented entirely within the READ, BREAD, GREAD, and WRITE states and must eventually reach the RACE state.

In regards to accuracy, there are many takeaways from Table II:

- We tested these tools over $580 \times 6 = 3480$ tests with 2076 injected races. HiRace caught all but 215 of these races—a "*completeness rate*" of 93%.
- All races missed by HiRace happened in executions with the smallest test graph used (5 nodes and 5 edges), where it is likely that the control-flow does not reach the data race. As discussed in Section I, Indigo's irregular code patterns are particularly well suited for testing this weakness of dynamic analysis tools.
- The "Sequential Comparison" results indicate the success rate of comparing GPU kernel results against a sequential implementation of the same algorithm. The fact that this naive testing method yields better results than state-of-the-art detection tools is distressing but further supports the value of HiRace's methodology.
- The reasons why other tools miss races in specific cases are unknown to us. Part of the reason is likely due to the eviction of access records as discussed above.
- In general, a compiler is *not required to compile a high-level program containing data races in any predictable manner* [2] (also known as "catch-fire semantics"). This raises doubts in our minds on whether tools that instrument compiled code are operating on valid programs when asked to check for races at the assembly level.

VI. Concluding Remarks

Whereas GPUs are powering much of the high-end HPC and ML revolution, there is a crucial omission in the arsenal of tools necessary to keep GPU programs trustworthy—namely *data-race checkers*. Given the growing uses of GPUs in safety-critical applications (e.g., automobile navigation systems) and even to implement various encryption schemes (fully homomorphic encryption or zero-knowledge proofs), and given the absence of a usable race checker for NVIDIA's GPUs, our work has the potential to plug a major hole in GPU program correctness checking.

We contribute a GPU data-race checker called HiRace that is sound (no unwarranted races reported) and (for all practical purposes) complete. All dynamic race checkers suffer from incompleteness due to schedule generation, that is, if the program contains data-dependent control flow, one must drive the code with sufficiently different test inputs to cover these paths. For instance, considering that HiRace's tests include graph benchmarks, one must, in principle, feed all graphs of all topologies to ensure completeness. There have been efforts to formally derive *cut-off bounds* to bound these tests [27]. Unfortunately, these works are unable to consider anything close to the kinds of problems we examine here.

Another type of completeness is based on *predictive race checking* [28], where one checks for races in one thread interleaving and predicts whether other interleavings contain races. These works largely apply to *mutex-based* synchronization patterns and are not applicable to GPU codes at the level discussed here (it may apply to codes that implement CUDA global barriers [29], which is of interest to us as future work).

In conclusion, we have tested HiRace on 3480 programs that are calibrated so that we can measure HiRace's bug-hunting efficiency. We will submit HiRace for artifact evaluation, thus plugging a serious hole in data-race checking of CUDA programs.

Future Work. There are a number of exciting directions for future work. First, an efficient schedule generation method can make HiRace more complete, hence more trustworthy. Second, if subwarp-level synchronization begins being used more, support for that in HiRace would be a good direction to pursue. Third, and most exciting of all, HiRace's technology is almost directly usable on AMD GPUs for which there is currently no race checker. Given the number of DOE labs that are investing heavily into AMD GPUs, we believe this would be of heightened interest to the community.

Acknowledgements

This work has been supported in part by the National Science Foundation under Awards 1955367, 1956106, 2124100, 2217154, and 2319507.

References

[1] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan, "Gklee: Concolic verification and test generation for gpus," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 215–224. [Online]. Available: https://doi.org/10.1145/2145816.2145844

[2] H.-J. Boehm and S. V. Adve, "You don't know jack about shared variables or memory models," *Commun. ACM*, vol. 55, no. 2, p. 48–54, feb 2012. [Online]. Available: https://doi.org/10.1145/2076450.2076465

[3] H.-J. Boehm, "How to miscompile programs with "benign" data races," in *3rd USENIX Workshop on Hot Topics in Parallelism (HotPar 11)*. Berkeley, CA: USENIX Association, May 2011. [Online]. Available: https://www.usenix.org/conference/hotpar-11/how-miscompile-programs-benign-data-races

[4] A. K. Kamath and A. Basu, "Iguard: In-gpu advanced race detection," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 49–65. [Online]. Available: https://doi.org/10.1145/3477132.3483545

[5] Nvidia, "cuda-memcheck," Website: https://docs.nvidia.com/cuda/cuda-memcheck/index.html, 2019.

[6] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, "Nvbit: A dynamic binary instrumentation framework for nvidia gpus," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 372–383. [Online]. Available: https://doi.org/10.1145/3352460.3358307

[7] A. Betts, N. Chong, A. Donaldson, S. Qadeer, and P. Thomson, "Gpuverify: a verifier for gpu kernels," in *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*, 2012, pp. 113–132.

[8] G. Li and G. Gopalakrishnan, "Scalable smt-based verification of gpu kernel functions," in *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 187–196. [Online]. Available: https://doi.org/10.1145/1882291.1882320

[9] Y. Peng, V. Grover, and J. Devietti, "CURD: a dynamic CUDA race detector," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, Philadelphia, PA, USA, June 18-22, 2018*, J. S. Foster and D. Grossman, Eds. ACM, 2018, pp. 390–403. [Online]. Available: https://doi.org/10.1145/3192366.3192368

[10] A. Eizenberg, Y. Peng, T. Pigli, W. Mansky, and J. Devietti, "Barracuda: Binary-level analysis of runtime races in cuda programs," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 126–140.

[11] P. Li, X. Hu, D. Chen, J. Brock, H. Luo, E. Z. Zhang, and C. Ding, "Ld: Low-overhead gpu race detection without access monitoring," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 1, mar 2017. [Online]. Available: https://doi.org/10.1145/3046678

[12] T. Cogumbreiro, J. Lange, D. Liew Zhen Rong, and H. Zicarelli, "Memory access protocols: Certified data-race freedom for gpu kernels," *FMSD*, 2023, 1 of 7 invited publications from CAV21. [Online]. Available: assets/faial-fmsd23.pdf

[13] C. Lattner, "LLVM and Clang – Advancing Compilers and Tools," in *Proceeding of the Free and Open Source Software Developers' European Meeting*, ser. FOSDEM '11, Feb. 2011.

[14] D. L. Dill, "The murphi verification system," in *International Conference on Computer Aided Verification*, 1996.

[15] J. R. Wilcox, C. Flanagan, and S. N. Freund, "Verifiedft: A verified, high-performance precise dynamic race detector," in *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 354–367. [Online]. Available: https://doi.org/10.1145/3178487.3178514

[16] Y. Liu, N. Azami, C. Walters, and M. Burtscher, "The indigo program-verification microbenchmark suite of irregular parallel code patterns," in *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2022, pp. 24–34.

[17] R. Netzer and B. P. Miller, *Detecting data races in parallel program executions*. University of Wisconsin-Madison, Computer Sciences Department, 1989.

[18] L. Lamport, "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Trans. Comput.*, vol. 28, no. 9, 1979.

[19] C. Flanagan and S. N. Freund, "FastTrack: Efficient and precise dynamic race detection," in *PLDI*, 2009, pp. 121–133.

[20] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71.

[21] S. Atzeni, G. Gopalakrishnan, Z. Rakamarić, D. H. Ahn, I. Laguna, M. Schulz, G. L. Lee, J. Protze, and M. S. Müller, "Archer: Effectively spotting data races in large OpenMP applications," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 53–62.

[22] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan, "GKLEE: concolic verification and test generation for GPUs," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012, New Orleans, LA, USA, February 25-29, 2012*, J. Ramanujam and P. Sadayappan, Eds. ACM, 2012, pp. 215–224.

[23] A. Donaldson, G. Gopalakrishnan, N. Chong, J. Ketema, G. Li, P. Li, A. Lokhmotov, and S. Qadeer, "Chapter 1 - formal analysis techniques for reliable gpu programming: current solutions and call to action," in *Advances in GPU Research and Practice*, ser. Emerging Trends in Computer Science and Applied Computing, H. Sarbazi-Azad, Ed. Boston: Morgan Kaufmann, 2017, pp. 3 – 21. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B978012803738600001X

[24] A. K. Kamath, A. A. George, and A. Basu, "Scord: A scoped race detector for gpus," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 1036–1049.

[25] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE international symposium on workload characterization (IISWC)*. Ieee, 2009, pp. 44–54.

[26] Y. Yu and S. T. Acton, "Speckle reducing anisotropic diffusion," *Trans. Img. Proc.*, vol. 11, no. 11, p. 1260–1270, nov 2002. [Online]. Available: https://doi.org/10.1109/TIP.2002.804276

[27] A. Kaiser, D. Kroening, and T. Wahl, "Dynamic cutoff detection in parameterized concurrent programs," in *Computer Aided Verification*, T. Touili, B. Cook, and P. Jackson, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 645–659.

[28] J. Roemer, K. Genç, and M. D. Bond, "High-coverage, unbounded sound predictive race detection," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 374–389. [Online]. Available: https://doi.org/10.1145/3192366.3192385

[29] T. Sorensen, A. F. Donaldson, M. Batty, G. Gopalakrishnan, and Z. Rakamaric, "Portable inter-workgroup barrier synchronisation for gpus," in *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016, part of SPLASH 2016, Amsterdam, The Netherlands, October 30 - November 4, 2016*, 2016, pp. 39–58. [Online]. Available: https://doi.org/10.1145/2983990.2984032